COMPUTER SCIENCE + COMPUTER ENGINEERING

Start Package 05

Volume 04 / continuation guide

Readiness, setup, exact commands and a first-session plan.

Modules 16–20 · Lessons 16.1–20.4 · Gate D

Package 05 · Expanded Volume 04 · September 28, 2026
Self-study teaching edition. Completion requires independent implementation, testing and review.

Volume 04: Computer Systems and Software Engineering

Continue at Lesson 16.1 after demonstrating the C++ ownership, data structures, algorithms and language-implementation work in Modules 11–15. This package expands Volume 4. It does not replace or silently skip the Volume 3 prerequisites.

The original program places **Modules 16–20 and Gate D** here. Its planning allowance is **380–600 hours**, including reading, exercises, measurements, integration and review. That is an author planning estimate, not a deadline or academic credit allocation.

Open these files

File	Use
STUDENT_HANDBOOK.pdf	Expanded lessons, worked reasoning, examples, practical checkpoints and quizzes.
LAB_WORKBOOK.pdf	The original lab briefs, concrete implementation/evidence steps, 50 extra exercises, readiness questions and module reviews.
ANSWER_KEY.pdf	Worked quiz, exercise, review and transfer answers. Keep it closed during an independent attempt.
CAPSTONE_SPEC.pdf	Gate D: requirements, resource bounds, failure cases, evidence and review rubric.
SYSTEMS_REFERENCE.pdf	Commands, units, synchronization, framing, persistence and troubleshooting notes.
COURSE.html	Offline browser course, searchable lessons and self-recorded progress.

Individual module PDFs are in `modules`. Editable documents and structured lesson data are in `source`. Student exercise files are in `native/student` and `python/student`; worked implementations are in the corresponding `reference` folders. Do not submit the reference implementation as independent work.

Readiness and tools

Complete the twelve readiness questions in the workbook. Repair gaps in lifetimes, unsigned arithmetic, build stages, exceptions, graph dependencies and benchmark reasoning before starting the associated lab. A strong readiness score is not permission to skip the practical evidence automatically.

Use the existing C++20 toolchain from Volume 3, CMake and a Python 3.10-or-newer interpreter with its `sqlite3` module. The executable teaching code uses Python standard-library facilities and the C++ standard library. No GPU, FPGA, paid cloud account, public server or new hardware is required. External readings still require internet access. The package does not install tools or change system settings.

The development validation used Python 3.13.5, GCC 14.2.0 and CMake 3.31.6 on Linux x86-64. These identify the tested environment, not the newest releases or the only possible configuration. Check `VALIDATION.md` for the complete results and platform limitations.

First session: establish a working baseline

Extract the ZIP and open a terminal in the `Game_Engineering_Package_05` directory. The following commands use `python3`; on Windows use your verified Python launcher, commonly `py -3`, instead.

```
python3 environment_check.py
cmake -S native -B build -DCMAKE_BUILD_TYPE=Debug
cmake --build build --config Debug --parallel 2
ctest --test-dir build -C Debug --output-on-failure
python3 tools/run_example.py 16.1 --build-dir build
```

`demo_16_1` should print `sum=23`. Use the runner rather than guessing where a multi-configuration generator placed the executable. It checks common Debug/Release executable locations.

Next, read Lesson 16.1 and annotate the fifteen-instruction routine at `native/asm/sum_u32_rv32.S`. The host example verifies an arithmetic contract; it does not emulate or execute RISC-V instructions. Save the compiler/environment record and your hand trace.

A useful first-session budget is 60–90 minutes: environment/build checks, a paper trace, one changed input, then the five questions. This is a suggested study block, not a requirement to finish the lesson in that time.

Check student work separately

```
python3 check_lab.py 16.1 --student
python3 check_lab.py 16.1 --reference
python3 check_lab.py 19.2 --student
```

Student checks initially fail with intentional TODO/NotImplemented errors. That is expected until the student implements the exercise. The `--reference` option checks a worked solution; its passing result does not mark the student lesson complete. Modules 16–19 have sixteen code-exercise interfaces; the four Module 20 labs require documents, review and release evidence rather than a function-only checker.

For a full reference check:

```
python3 tools/verify.py
```

Without a build-directory argument this creates a fresh build under `.build`, runs native/Python checks, verifies all twenty example outputs and runs the finite local capstone demo. It does not download dependencies. On platforms without permitted loopback sockets, record the integration test as blocked rather than treating it as passed.

Progress and assessment

Each expanded lesson has five questions: the three original questions plus two application questions. Score each 0–2: incorrect/absent, partial, or correct with reasoning. Suggested formative pass: **9/10**, the required practical evidence, assigned reading and an independently solved transfer task. A 9/10 total also requires at least 5/6 across the retained original questions.

The browser records reading, lab completion, transfer completion and a manually entered quiz score. Its completion label is a self-recorded status, not an automated competence certificate. Export progress before moving the HTML file or changing browsers; storage availability varies.

Working safely and accurately

Use synthetic inputs, private scratch folders and ordinary user privileges. Do not change kernel priorities, flush system caches, disable protective settings or run stress workloads on production data. The gateway binds only to `127.0.0.1`; keep it that way. No authentication, encryption or hostile-executable sandbox is supplied.

Native C++ concurrency is taught with C++ atomics, mutexes and threads. Python orchestration is a separate layer and is not presented as proof of the C++ memory model. SQLite is the executable no-server database backend; PostgreSQL is the assigned comparison reading and optional SQL companion, not an untested compatibility claim.

Preserve the difference between a model prediction, an observed benchmark, a sanitizer result and a proof under assumptions. These evidence categories are used throughout the package.